AI-POWERED ROAD HAZARD DETECTION SYSTEM

AI Analysis Engine — Technical Report

Real-Time CCTV Analysis | Video Upload Analysis | Mathematical Formulations

Version	1.0
Document Type	Technical Specification
Stack	YOLOv8 + FastAPI + OpenCV

PROBLEM STATEMENT — PS4: AI-Based Road Hazard Detection

1. Executive Summary

This document defines the complete AI analysis engine for the Road Hazard Detection System — covering two primary operating modes: Real-Time CCTV Analysis and Video Upload Analysis. It provides all mathematical formulations, scoring functions, detection pipelines, and analytical workflows required for implementation.

Component	Description
Detection Model	YOLOv8n (default) / YOLOv8s (accuracy mode)
Hazard Classes	Potholes, Stray Animals, Vehicles, Humans (accident context)
Analysis Modes	Real-Time Stream (RTSP/CCTV) + Video Upload (batch)
Scoring Engine	Severity Score, Risk Score, Priority Index — all mathematically defined
Output	Structured JSON incidents, annotated frames, dashboard alerts, email triggers

2. Two Analysis Modes Overview

2.1 Mode A — Real-Time CCTV Analysis

In real-time mode, the system continuously reads frames from a live CCTV stream (RTSP feed or equivalent). Each frame is processed through the YOLO detection pipeline, annotated with bounding boxes and labels, and the result is streamed back to the monitoring dashboard. Incidents are raised when confirmed hazard conditions are met across multiple consecutive frames.

Flow: RTSP Stream → OpenCV Frame Capture → YOLOv8 Detection → Annotation → Severity Scoring → Incident Creation → WebSocket Push → Dashboard Display

Real-Time Processing Parameters

Parameter	Value	Rationale
Frame Rate	5–10 FPS (configurable)	Balance between detection latency and CPU usage
Frame Skip	Every 2nd–3rd frame skipped	Reduces load while maintaining motion coverage
Detection Window	Rolling 5-frame buffer	Used for temporal confirmation and motion analysis
Confirmation Threshold	3 of 5 consecutive frames	Prevents single-frame false positives
Latency Target	< 500ms end-to-end	Required for real-time alerting
WebSocket Push	Per incident creation	Triggers admin dashboard update immediately

2.2 Mode B — Video Upload Analysis

In video upload mode, a user submits a pre-recorded video file through the testing dashboard. The system processes the entire video at a controlled frame rate, runs full hazard detection, and generates a structured analysis report with annotated clips, event timeline, and statistical summary.

Flow: Upload → Frame Extraction (OpenCV) → Batch YOLOv8 Detection → Event Assembly → Severity Scoring → Report Generation → Dashboard Display + Download

Video Upload Processing Parameters

Parameter	Value	Rationale
Supported Formats	MP4, AVI, MOV, MKV	Common dashcam and CCTV export formats
Max Upload Size	500 MB (demo), 2 GB (production)	Constrained by storage and processing time
Processing FPS	1–5 FPS (batch mode)	Slower processing acceptable; accuracy prioritized
Output	Annotated frames + JSON events + summary	Full traceability of each detection
Timeline Resolution	Per-frame timestamp	Precise event localization within video
Report Format	JSON + downloadable HTML/PDF	Shared with authorities for incident record

3. YOLO Detection Engine

3.1 Model Selection and Class Mapping

The system uses Ultralytics YOLOv8. YOLOv8n is the default for speed. YOLOv8s is available as an accuracy upgrade. A custom fine-tuned model handles pothole detection since potholes are not in the standard COCO dataset.

Detection Class	Source Model	COCO Class ID	Custom Label
Pothole	Fine-tuned YOLOv8	—	pothole
Dog	YOLOv8 COCO	16	dog
Cat	YOLOv8 COCO	15	cat
Cow	YOLOv8 COCO	19	cow
Horse	YOLOv8 COCO	17	horse
Vehicle (Car)	YOLOv8 COCO	2	car
Vehicle (Truck)	YOLOv8 COCO	7	truck
Vehicle (Motorbike)	YOLOv8 COCO	3	motorcycle
Person	YOLOv8 COCO	0	person

3.2 Core Detection Formula — IoU (Intersection over Union)

IoU is the fundamental metric used by YOLO to evaluate the quality of bounding box predictions against ground truth. It is also used in NMS and tracking.

```
IoU(A, B) = Area(A ∩ B) / Area(A ∪ B)
```

Where A is the predicted bounding box and B is the ground truth box. IoU ranges from 0 (no overlap) to 1 (perfect overlap). A detection is considered valid when IoU ≥ threshold (typically 0.45 for NMS, 0.5 for evaluation).

3.3 YOLO Confidence Score

Each detected object is assigned a confidence score combining objectness probability and class probability:

```
Confidence = P(Object) × P(Class | Object) = P_obj × P_cls
```

Filter Rule: Detections with Confidence < 0.45 are discarded. Hazard incidents require Confidence >= 0.55. High-severity alerts require Confidence >= 0.70.

3.4 Non-Maximum Suppression (NMS)

NMS prevents duplicate detections of the same object by suppressing lower-confidence overlapping boxes:

```
Keep box B_i if: Confidence(B_i) > Confidence(B_j) for all j where IoU(B_i,
B_j) >= NMS_threshold
```

NMS threshold is set to 0.45. This is critical for road scenes with dense vehicle clusters and overlapping pothole regions.

3.5 Normalized Bounding Box Coordinates

YOLO outputs normalized coordinates relative to image dimensions. These must be converted for pixel-space operations:

```
x_pixel = x_norm × img_width
y_pixel = y_norm × img_height
w_pixel = w_norm × img_width
h_pixel = h_norm × img_height
Bounding Box Area = w_pixel × h_pixel
```

4. Severity Scoring — Mathematical Definitions

4.1 Pothole Severity Score

Pothole severity is computed as a weighted combination of bounding box area (normalized), detection confidence, road zone position, and repeated observation frequency.

Pothole Severity Score Formula

```
SS_pothole = w1 × A_norm + w2 × C + w3 × Z + w4 × F_repeat
```

Variable	Definition	Formula / Value
SS_pothole	Pothole Severity Score (0–100)	Composite weighted score
A_norm	Normalized bounding box area	A_norm = (w_px × h_px) / (img_w × img_h)
C	YOLO detection confidence	Output from YOLOv8, range [0,1]
Z	Road zone position weight	Centre lane = 1.0, edge = 0.5, offroad = 0.2
F_repeat	Repeat detection frequency	F = detections_in_last_10_frames / 10
w1	Area weight	0.35
w2	Confidence weight	0.30
w3	Zone position weight	0.20
w4	Frequency weight	0.15

Scale Rule: SS_pothole × 100 maps to 0–100. Low: 0–33, Medium: 34–66, High: 67–100.

Road Zone Classification

```
Zone = 'CENTRE' if (img_width/4 < x_centre < 3×img_width/4) else 'EDGE' or
                                                    'OFF_ROAD'
```

4.2 Pothole Area Growth Rate (Before/After Tracking)

For recurring pothole detections, the system tracks the area change across time to determine whether the pothole is worsening, stable, or resolved.

```
Growth_Rate = (A_after - A_before) / A_before × 100  (percentage change)
Status = 'GROWING'   if Growth_Rate > +15%
Status = 'STABLE'    if -15% <= Growth_Rate <= +15%
Status = 'RESOLVED'  if Growth_Rate < -80% (pothole no longer detected)
```

4.3 Animal Risk Score

Animal risk is computed from detection confidence, proximity to the road zone, behavior classification, and motion velocity estimate.

Animal Risk Score Formula

```
RS_animal = w1 × C + w2 × P_road + w3 × B_weight + w4 × V_norm
```

Variable	Definition	Value / Formula
RS_animal	Animal Risk Score (0–100)	Composite weighted score × 100
C	Detection confidence	YOLOv8 output, range [0,1]
P_road	Road proximity weight	On road = 1.0, Approaching = 0.6, Roadside = 0.3
B_weight	Behavior risk weight	Running/crossing = 1.0, Moving = 0.6, Stationary = 0.2
V_norm	Normalized velocity estimate	V_norm = pixel_disp / (frame_interval × img_w)
w1	Confidence weight	0.25
w2	Proximity weight	0.35
w3	Behavior weight	0.30
w4	Velocity weight	0.10

Behavior Classification Logic

```
B_class = 'RUNNING'   if V_pixel > V_threshold_high   (V_thresh_high = 20 px/frame)
B_class = 'CROSSING'  if trajectory_angle intersects road_boundary
B_class = 'MOVING'    if V_pixel > V_threshold_low   (V_thresh_low = 5 px/frame)
B_class = 'STATIONARY' if V_pixel <= V_threshold_low
```

4.4 Accident Severity Score

Accident severity is derived from vehicle overlap (collision proxy), motion anomaly magnitude, number of vehicles involved, human presence, and estimated speed before impact.

Accident Severity Score Formula

```
SS_accident = w1 × IoU_collision + w2 × M_anomaly + w3 × N_vehicles_norm + w4 × H_pres + w5 × S_norm
```

Variable	Definition	Value / Formula
SS_accident	Accident Severity Score (0–100)	Composite × 100
IoU_collision	IoU between colliding vehicle bounding boxes	Range [0,1]; > 0.3 = contact
M_anomaly	Motion anomaly score	See Section 4.5
N_vehicles_norm	Normalized involved vehicle count	N_vehicles / max_expected_vehicles (= 5)
H_pres	Human presence near accident zone	1.0 if person detected within zone, else 0.0
S_norm	Normalized pre-collision speed estimate	S_norm = V_pre / V_max (V_max = 120 km/h proxy)
w1	Collision weight	0.35
w2	Motion anomaly weight	0.25
w3	Vehicle count weight	0.15
w4	Human presence weight	0.15
w5	Speed weight	0.10

Score Range	Severity Label	Action
0 – 30	LOW	Log and monitor; no immediate dispatch
31 – 70	MEDIUM	Alert dashboard; schedule response
71 – 100	HIGH	Immediate email + dashboard alert; dispatch

5. Motion Analysis Formulas

5.1 Centroid Tracking

Object tracking across frames uses centroid-based matching. The centroid is computed from bounding box coordinates:

```
Centroid = ((x + w/2), (y + h/2))
Distance(C1, C2) = sqrt((x2-x1)^2 + (y2-y1)^2)   [Euclidean]
```

Objects within a distance threshold (D_max = 50 pixels) between frames are matched as the same tracked object. If no match is found within D_max, the object is treated as new.

5.2 Optical Flow — Motion Anomaly Score

The motion anomaly score uses dense optical flow (Lucas-Kanade or Farneback) to detect sudden unusual motion patterns indicative of accidents:

```
Flow_magnitude(x,y) = sqrt(u(x,y)^2 + v(x,y)^2)
```

```
Mean_flow = (1/N) × Σ Flow_magnitude(x,y)  for all N pixels in ROI
σ_flow = sqrt((1/N) × Σ (Flow_magnitude - Mean_flow)^2)  [Standard deviation]
M_anomaly = min(1.0, σ_flow / σ_baseline)
```

σ_baseline: The baseline standard deviation of flow is computed as a rolling average over 30 normal frames at session start. Sudden spikes (σ_flow >> σ_baseline) indicate collision or abrupt motion.

5.3 Pixel Displacement Velocity

To estimate object velocity in pixel-space (used for both animal behavior and accident speed proxy):

```
V_pixel(t) = Distance(Centroid(t), Centroid(t-1)) / frame_interval
V_pixel_avg = (1/n) × Σ V_pixel(t)  for last n frames (n=5 default)
V_kmh_approx = V_pixel × (known_road_width_m / road_width_pixels) × FPS × 3.6
```

5.4 Sudden Stop Detection

Hit-and-run and accident detection relies on identifying vehicles that suddenly decelerate:

```
Deceleration(t) = V_pixel(t-1) - V_pixel(t)
Sudden_Stop = True  if Deceleration(t) > V_pixel(t-1) × 0.7  (70% velocity drop in 1 frame)
Hit_and_Run_Flag = True  if Sudden_Stop AND vehicle exits frame within 5 frames
```

5.5 Traffic Density and Congestion Level

Congestion level is computed as a normalized vehicle density relative to the road capacity estimate:

```
Traffic_density = N_vehicles_in_ROI / ROI_area_pixels
Congestion_Level = min(1.0, Traffic_density / Traffic_density_max)
Congestion_Label: LOW (<0.33), MEDIUM (0.33–0.66), HIGH (>0.66)
```

6. Priority Index — Unified Ranking Formula

6.1 Incident Priority Index (PI)

The Priority Index is a single number that ranks all active incidents from all hazard types to help authorities triage response actions. It combines severity, confidence, recency, and hazard category weight.

Priority Index Formula

```
PI = w_type × SS_normalized × C × T_decay
```

Variable	Definition	Value / Formula
PI	Priority Index (0–100)	Output ranking score
w_type	Hazard type weight	Accident = 1.0, Animal = 0.8, Pothole = 0.6
SS_normalized	Severity score normalized to [0,1]	SS / 100
C	Detection confidence	YOLOv8 output [0,1]
T_decay	Time decay factor	T_decay = exp(-λ × Δt) where Δt = seconds since detection
λ (lambda)	Decay rate	0.001 (slow decay over 15 min to near 0.4)

Usage: All open incidents are sorted by PI descending. The top-K incidents are surfaced in the priority alert panel on the authority dashboard. PI is recalculated every 30 seconds.

```
T_decay = e^(-0.001 × Δt)    [After 15 min: decay ≈ 0.41; after 60 min: decay ≈ 0.05]
```

6.2 Daily Summary Analytics

The following aggregate metrics are calculated for the analytics dashboard:

```
Incident_Rate(day) = Total_incidents(day) / Total_hours_monitored

Severity_Distribution(type) = Count(type, severity) / Total_incidents(type) × 100

Resolution_Rate = Resolved_incidents / Total_incidents × 100

Mean_Detection_Confidence = (1/N) × Σ Confidence_i

Hotspot_Score(camera) = Σ SS_i × w_type_i  for all incidents at camera in window
```

7. Night-Time Detection Pipeline

7.1 Low-Light Preprocessing Formulas

Before YOLO inference on night-time frames, preprocessing enhances visibility using histogram equalization and contrast adjustment:

CLAHE (Contrast Limited Adaptive Histogram Equalization)

```
L_enhanced = CLAHE(L_channel, clip_limit=2.0, tile_grid=(8,8))
```

Applied only to the L (luminance) channel after converting BGR to LAB color space. This preserves color while boosting dark regions.

Gamma Correction

```
I_out = I_in^(1/γ)  where γ < 1 brightens dark images
Recommended: γ = 0.5 for night scenes, γ = 0.7 for overcast/dusk
```

Noise Reduction

```
I_denoised = GaussianBlur(I, kernel=(3,3), σ=0)  OR  fastNlMeansDenoising(I,
                                                h=10)
```

7.2 Adaptive Confidence Threshold

Night-time inference uses lower confidence thresholds to compensate for reduced feature visibility:

```
C_threshold_night = C_threshold_day × 0.80
```

For example: Day threshold 0.55 → Night threshold 0.44. This reduces missed detections at the cost of marginally more false positives, which are filtered by temporal confirmation.

8. Dashboard Graph Analytics — Computed Metrics

8.1 Graphs Displayed in the Authority Dashboard

All graphs are computed server-side and refreshed on a polling interval. The mathematical basis for each graph type is defined below.

Graph	X-Axis	Y-Axis	Formula / Source
Incident Rate Over Time	Time (hour/day)	Incidents per hour	Count of incidents grouped by time bucket
Severity Distribution	Severity Level (Low/Med/High)	Count / Percentage	Count(severity_label) / Total × 100
Hazard Type Breakdown	Hazard Type	Count	Count(hazard_type) grouped by type
Resolution Rate Trend	Date	% Resolved	Resolved / Total per day × 100
Hotspot Heatmap (Leaflet)	Latitude / Longitude	Intensity	Hotspot_Score per camera location
Confidence Distribution	Confidence Bin (0–1)	Count	Histogram of confidence scores, bin=0.05
Pothole Growth Tracking	Time	Bounding Box Area	A_normalized per re-detection event

Graph	X-Axis	Y-Axis	Formula / Source
Animal Activity Timeline	Hour of Day	Risk Score	Mean RS_animal per hour across rolling window
Accident Frequency Map	Camera ID	Accidents/day	Count per camera per day
Priority Index Trend	Time	PI Score	PI recalculated per incident over time

8.2 Real-Time Frame Overlay Annotations

For both analysis modes, each output frame includes the following annotations rendered with OpenCV:

Annotation	Color	Content
Bounding Box	Red (#FF0000)	Detected hazard outline (pothole, animal, collision zone)
Vehicle Box	Blue (#0000FF)	Each tracked vehicle with ID label
Person Box	Yellow (#FFFF00)	Human detection
Severity Label	White on colored background	HIGH / MEDIUM / LOW with score
Confidence	White text	e.g., conf: 0.87
Track ID	Cyan text	Persistent object ID across frames
Velocity	Green text	km/h estimate overlaid near vehicle
Zone Boundary	Semi-transparent overlay	Road zone polygon marking
Timestamp	White, top-left	Frame timestamp or video timestamp

9. AI Service API Contracts

9.1 Real-Time Frame Submission (WebSocket)

Client sends frame over WebSocket. Server returns annotated frame and detection payload.

Client → Server: { "frame": "<base64_encoded_jpeg>", "camera_id": "CAM_01", "timestamp": 1711449600.5 }

Server → Client: { "annotated_frame": "<base64>", "detections": [{ "class": "pothole", "confidence": 0.87, "bbox": [x,y,w,h], "severity_score": 78, "severity_label": "HIGH" }], "incident_created": true, "incident_id": "uuid" }

9.2 Video Upload REST Endpoint

POST /api/upload: Accepts multipart form-data with video file. Returns job_id. Processing runs asynchronously.

GET /api/upload/{job_id}/status: { "status": "processing", "progress": 62, "frames_processed": 310, "frames_total": 500 }

GET /api/upload/{job_id}/report: { "summary": { "total_incidents": 4, "potholes": 2, "animals": 1, "accidents": 1 }, "events": [...], "timeline": [...] }

10. Implementation Checklist

#	Task	Module	Status
1	YOLOv8n model load and inference on single frame	detector.py	[ ]
2	Confidence filter (< 0.45 discard)	detector.py	[ ]
3	NMS applied (threshold 0.45)	detector.py	[ ]
4	Bounding box pixel conversion	detector.py	[ ]
5	Centroid tracker implementation	tracking.py	[ ]
6	Pothole severity formula (SS_pothole)	pothole_engine.py	[ ]
7	Pothole zone classification	pothole_engine.py	[ ]
8	Pothole before/after area tracking	pothole_engine.py	[ ]
9	Animal behavior classification	animal_engine.py	[ ]
10	Animal risk score formula (RS_animal)	animal_engine.py	[ ]
11	Optical flow motion anomaly	accident_engine.py	[ ]
12	Vehicle IoU collision detection	accident_engine.py	[ ]
13	Sudden stop and hit-and-run detection	accident_engine.py	[ ]
14	Accident severity formula (SS_accident)	accident_engine.py	[ ]
15	Priority Index computation	scoring.py	[ ]
16	Time decay function for PI	scoring.py	[ ]
17	CLAHE night-time preprocessing	detector.py	[ ]
18	OpenCV frame annotation (all overlays)	detector.py	[ ]
19	WebSocket frame stream (real-time mode)	app.py	[ ]
20	Video upload endpoint + async job queue	app.py	[ ]
21	Incident JSON → PostgreSQL (Neon)	app.py	[ ]
22	Email alert trigger (High severity)	app.py	[ ]

#	Task	Module	Status
23	Analytics aggregate queries	app.py	[ ]
24	Dashboard graph data endpoints	app.py	[ ]

11. Formula Quick Reference Sheet

All Core Formulas — At a Glance

```
IoU(A,B) = Area(A∩B) / Area(A∪B)
Confidence = P_obj × P_cls
A_norm = (w_px × h_px) / (img_w × img_h)
SS_pothole = 0.35×A_norm + 0.30×C + 0.20×Z + 0.15×F_repeat  [×100]
Growth_Rate = (A_after - A_before) / A_before × 100
RS_animal  = 0.25×C + 0.35×P_road + 0.30×B_weight + 0.10×V_norm  [×100]
SS_accident = 0.35×IoU + 0.25×M_anomaly + 0.15×N_norm + 0.15×H + 0.10×S_norm
[×100]
M_anomaly  = min(1.0, σ_flow / σ_baseline)
V_pixel(t) = Distance(C_t, C_{t-1}) / frame_interval
Sudden_Stop = True if Deceleration > 0.7 × V_prev
PI = w_type × (SS/100) × C × e^(-0.001 × Δt)  [×100]
Hotspot_Score = Σ SS_i × w_type_i  for camera window
Resolution_Rate = Resolved / Total × 100
C_threshold_night = C_threshold_day × 0.80
Gamma Correction: I_out = I_in^(1/γ)
```

12. Technology Stack Summary

Layer	Technology	Purpose
AI Detection	Ultralytics YOLOv8n / YOLOv8s	Object detection — potholes, animals, vehicles, persons
Video Processing	OpenCV (cv2)	Frame capture, optical flow, annotation, preprocessing
AI Service API	Python 3.11 + FastAPI	REST + WebSocket endpoints for both analysis modes
ML Tracking	Custom centroid tracker	Cross-frame object identity and motion measurement

Layer	Technology	Purpose
Database	Neon PostgreSQL + Prisma ORM	Incident storage, analytics queries, camera registry
Frontend Dashboard	Next.js 14 + TypeScript	Authority dashboard + testing dashboard (video upload)
UI Components	Tailwind CSS + shadcn/ui	Consistent, responsive design
Charts	Recharts	All analytics graphs (severity, rate, distribution)
Map	Leaflet + OpenStreetMap	Hotspot heatmap, camera geo-locations
Real-Time Updates	WebSocket / Server-Sent Events	Live frame streaming and incident push
Email Alerts	Nodemailer + SMTP	High-severity authority notification
Storage	Local filesystem (dev) / Cloud (prod)	Snapshots, annotated clips, report files

AI Road Hazard Detection — Technical Analysis Report

All formulas, thresholds, and weights defined in this document are implementation baselines and may be tuned based on observed performance in the target deployment environment.

PS4: AI-Based Road Hazard Detection | Version 1.0 | Free & Open-Source Stack